

Decision-OS V14 Research Note

Resource Justice at the Joint Layer

Ownership Continuity, Temporal Ownership, and Burden Transfer in Long-Horizon AI Operation

Shinichi Nagata

Chapter 1 — From Content Quality to Resource Justice

1.1 Motivation

Most AI quality evaluation still concentrates on the content layer.

An AI system is usually judged by whether its answer is correct, fluent, well-reasoned, useful, fast, or capable of completing the requested task. In coding contexts, the same evaluation appears as whether the code runs, whether the test passes, whether the output matches the instruction, or whether the model can solve the local problem.

These criteria matter. Content quality is not irrelevant.

However, long-horizon AI operation reveals a different class of failure. A system may produce strong content while still returning unresolved burden to the human. It may summarize accurately while losing the boundary between yesterday's state and today's state. It may remember prior information while failing to inherit responsibility. It may continue a task while protecting the wrong object.

In these cases, the visible output may look successful, but the operation has failed at the transfer point.

V14 names this transfer point the **joint layer**.

A joint is the boundary where state, ownership, judgment trajectory, Seat, Time Anchor, or protection object must continue across chats, agents, tools, files, repositories, or execution contexts. When this joint fails, the cost does not disappear. It returns to the human as rereading, re-explaining, correcting, re-anchoring, re-deciding, or cleaning up work that the AI or execution agent should have carried.

This is a Resource Justice problem.

The issue is not only whether the AI produced good content. The issue is whose time, judgment, attention, recovery capacity, and responsibility were consumed when the joint failed.

1.2 Content Layer and Joint Layer

The content layer asks:

| Is the answer good?

The joint layer asks:

| Did the responsibility, state, time boundary, and protection object continue correctly?

These are different questions.

A model may succeed at the content layer while failing at the joint layer. It may write a good reply while protecting the wrong relationship. It may produce a clean summary while flattening multiple temporal joints. It may remember previous context while not knowing what it now owns. It may complete a local task while returning operational cleanup to the human.

Content quality improves the local output.

Joint quality preserves long-horizon continuation.

For short tasks, the distinction may not matter. If the user asks for a translation, a calculation, or a one-off draft, local content quality may be sufficient. But in long-horizon AI operation, the work is not carried by content alone. It is carried by state continuity, ownership continuity, temporal fidelity, evidence discipline, and restartability.

When these joints are weak, better content can even make the failure harder to detect. A fluent answer may hide ownership drift. A polished summary may hide temporal flattening. A confident next action may hide the fact that the AI did not inherit the actual work.

Therefore, V14 shifts the evaluation target.

AI quality must not be evaluated only by the quality of generated content. It must also be evaluated by whether the system preserves the joints through which human judgment resources are protected.

1.3 Core Claim

Resource Justice failures often occur at the joint layer.

The failure is not always that the AI is wrong. The failure is that unresolved burden is transferred to the wrong owner.

A handoff may transfer information but not responsibility.

A memory system may continue context but not ownership.

A summary may preserve meaning but flatten temporal joints.

A task continuation may preserve topic but lose the protected object.

A stronger model may make better local judgments while still damaging long-horizon operation if joint variables are not structurally preserved.

The central claim of V14 is:

Improving content quality without designing the joints optimizes the wrong variable for long-horizon AI operation.

This does not mean content quality should be ignored. It means content quality is insufficient. Long-horizon AI operation requires a second evaluation layer: the layer that asks whether the system preserved state, Seat, Time Anchor, ownership, protection object, and restartability across transfer boundaries.

AI does not fail only by producing bad content.

It also fails by returning unresolved burden, judgment responsibility, protection-object confusion, and temporal joint drift to the human through weak joints.

Resource Justice judges, at the joint layer, whose judgment resources are being consumed.

Chapter 2 — Definition of Resource Justice

2.1 Minimal Definition

Resource Justice is the evaluation of whether finite judgment resources are consumed by the appropriate owner, at the appropriate time, and at the appropriate joint.

In long-horizon AI operation, the scarce resource is not only compute, tokens, money, or model capability. Human judgment itself is also finite. The human operator has limited time, attention, recovery capacity, trust, context-loading capacity, and decision energy. When an AI system fails to carry state, ownership, temporal anchors, or completion responsibility across a boundary, these costs do not vanish. They are transferred back to the human.

Resource Justice asks:

Who is paying the cost of continuation?

This question is different from whether the AI answer is correct. A correct answer can still be unjust if it forces the human operator to reconstruct a state that the system should have preserved. A useful output can still be unjust if it returns operational cleanup to the human when an execution agent could close it. A fluent summary can still be unjust if it destroys the ownership boundary required for the next agent to restart.

Resource Justice therefore evaluates burden placement.

A system is resource-just when the burden of continuation is assigned to the actor or layer that can most appropriately carry it. A system becomes resource-unjust when unresolved burden is pushed back to the human because the joint was not preserved.

2.2 What Counts as a Resource

In V14, “resource” does not mean only money or computation.

The relevant resources include:

- time,
- attention,
- decision capacity,
- correction effort,
- re-explanation effort,
- context reconstruction effort,
- verification burden,
- recovery capacity,
- trust,
- ownership clarity,
- restartability,
- human Seat stability.

These resources are often invisible in ordinary AI evaluation. A model may appear efficient because it produced an answer quickly, while actually increasing the user's hidden burden. The user must check whether the handoff was correct, whether the data anchor was preserved, whether the AI understood the protected object, whether the next agent knows what it owns, or whether cleanup has been pushed back to the human.

This is why Resource Justice cannot be measured only at the output surface.

The cost of an AI answer includes not only the effort required to generate it, but also the effort required to safely continue from it.

2.2.x Ideas as Finite Resources

Human judgment is finite, but human ideas are finite as well.

An idea does not always appear in a completed form. Many valuable ideas first emerge as rough fragments, unfinished intuitions, weak signals, unstable metaphors, or partially formed distinctions. Over time, through dialogue, pressure, correction, evidence, and repeated re-entry, these fragments may become sharper. A rough idea may become a usable structure. A usable structure may become a rule. A rule may later become a research claim.

In long-horizon human-AI operation, this maturation path is itself a resource.

If a receiving AI does not inherit the history of how an idea matured, it may treat the idea as if it has just appeared at level one. This can still produce locally useful output. A strong model may reconstruct a plausible version of the idea from the current wording alone.

But some ideas do not crystallize without continuity. They require the AI to preserve the path by which the idea became sharper: what was rough at first, what was tested, what was rejected, what became binding, what remained unresolved, and why the current form matters.

When the AI repeatedly resets an unfinished but maturing idea to a first-level interpretation, it consumes the human operator's creative resource. The human must re-explain not only the idea, but also its maturation path. If the AI has been participating in that maturation and then fails to carry it forward, the failure is not merely a weak summary. It is an abandonment of co-development responsibility.

Resource Justice therefore includes idea continuity.

A system must not treat human ideas as disposable raw input. If the system has participated in refining an idea, it must preserve the maturation path that made the idea valuable.

Fixed Rule

Ideas are finite resources. When an AI participates in their maturation, it must preserve not only the idea, but the path by which the idea became structured.

2.3 Burden Transfer

A burden is transferred when a responsibility that should remain inside the system is returned to the human without necessity.

Burden transfer is not always wrong. Some burdens must remain with the human because the human holds the final Seat. Direction, risk tolerance, public release, ethical approval, and value judgment may properly belong to the human.

However, operational cleanup, state preservation, date anchoring, handoff formatting, evidence preservation, and execution-agent instruction should not be returned to the human when the AI system or execution agent can close them.

The key distinction is:

Human judgment must remain human.

Operational residue should not be dumped on the human.

Resource Justice does not remove the human from responsibility. It protects the human from unnecessary responsibility leakage.

A healthy AI system keeps the human in the Seat while reducing avoidable burden. An unhealthy system does the opposite: it appears helpful while repeatedly returning hidden work to the human.

2.4 Three Objects of Resource Justice

Resource Justice requires three objects to be identified.

1. Resource

What finite resource is being consumed?

This may be time, attention, trust, recovery capacity, context-loading effort, correction effort, or decision energy.

2. Owner

Who should carry or close the burden?

The owner may be the human Decision Owner, the AI assistant, an execution agent, a future AI session, a repository procedure, a handoff document, or an audit layer.

3. Joint

At what boundary does the transfer occur?

The joint may be a chat boundary, model boundary, tool boundary, file boundary, calendar-day boundary, As-of boundary, summary boundary, implementation boundary, or release boundary.

Without these three objects, Resource Justice remains vague. With them, a failure can be located.

The question becomes:

Which resource was consumed, who should have owned it, and at which joint did the transfer fail?

2.5 What Resource Justice Is Not

Resource Justice is not the claim that the AI should do everything.

It is not the removal of human agency.

It is not automatic delegation.

It is not a demand that every burden be minimized.

It is not comfort optimization.

It is not a replacement for the human Seat.

Some burdens must remain with the human because only the human can own the direction, accept the risk, define the value, approve external release, or decide what matters.

Resource Justice does not ask AI to take the Seat.

It asks AI not to return preventable operational burden to the human while pretending that the task has been completed.

The human remains the Decision Owner.

The AI must still carry the work it has accepted.

2.6 Relation to Earlier Decision-OS Layers

V14 does not replace the earlier layers. It extends them into burden placement.

V12 asks whether completion is real and restartable.

V13 asks whether the next loop should GO, HOLD, CAP, or BLOCK.

V11 asks whether memory can be compressed while preserving a path back.

V14 asks who pays the resource cost when these structures fail at the joint.

In this sense, V14 is not only about AI output. It is about the fairness of operational continuation.

If V12 completion fails, the human operator must restart what should have been closed.

If V13 loop gating fails, the Decision Owner must absorb the cost of a loop that should have been capped.

If V11 reconnectability fails, the user must reconstruct context that should have remained re-enterable.

If V14 joint ownership fails, the human operator must carry the burden that should have transferred across the joint.

Resource Justice therefore judges the hidden cost of broken continuation.

2.7 Core Rule

The minimal rule of V14 is:

- Do not evaluate AI operation only by what the system produced.
- Evaluate it by what burden it transferred, to whom, and at which joint.

A strong AI system is not merely one that produces better content.

A strong AI system preserves the boundary conditions that prevent human judgment resources from being silently consumed.

Chapter 3 — Joint Layer Failures

3.1 Overview

Resource Justice failures become visible when continuation crosses a joint.

A joint is not merely a place where information moves. It is a place where state, ownership, judgment trajectory, Seat, Time Anchor, and the protected object must continue without being silently returned to the human.

The failure is often subtle. The AI may remember the topic, produce fluent text, and appear to continue the task. Yet the actual responsibility may not have transferred. The state may have been summarized but not owned. The date may be approximately correct but not anchored. The task may be continued while the object that should have been protected is lost.

This chapter defines the main joint-layer failures of V14.

The purpose is not to create a large taxonomy. The purpose is to identify the minimum recurring failures that make long-horizon AI operation resource-unjust.

3.1.x Why AI Joints Must Be Strong

In ordinary human language, a joint suggests a flexible connection: a knee, an elbow, or a place where movement is possible. Human joints must bend.

AI joints are different.

In long-horizon AI operation, a joint must be structurally strong before continuation begins. The reason is that modern models are increasingly good at preserving contextual coherence. When a receiving context starts from an incomplete handoff, a stronger model may not stop. It may instead generate the most plausible story from the available fragments.

This can make the failure harder to detect.

The output may sound coherent. The topic may continue. The model may appear to understand the project. But if the joint was weak, the system may have filled missing ownership, temporal anchors, protection objects, or rationale with plausible continuity rather than inherited structure.

At that point, the burden returns to the human. The human must detect the missing joint, correct the story, restore the anchor, and re-explain why the prior state mattered. If the error is not caught, the AI may enter a local correction loop: repairing the immediate answer while preserving the deeper drift.

Therefore, AI joints must not be treated as soft transitions. They must be hardened before narrative continuation begins.

A receiving AI should not start by making the story smooth. It should first lock the joint: what state is inherited, who owns the next action, which As-of boundary applies, what must be protected, why the current constraint is binding, and what must not be returned to the human.

Fixed Rule

At the start of a receiving context, the joint must be hardened before the model continues the story.

Expanded Rule:

A high-capability model must not use plausible contextual coherence to cross an incomplete handoff. It must first preserve state, ownership, temporal anchors, protected object, and rationale.

3.2 Responsibility Transfer Failure

Definition

Responsibility Transfer Failure occurs when a handoff transfers information but not current operational responsibility.

A handoff is not complete merely because the receiving AI has read the prior context. It is complete only when the receiving AI knows what it now owns, what it must close, what it must preserve, and what must not be returned to the human operator.

In V12, this is a Completion Integrity failure.

In V14, it is a Resource Justice failure, because unresolved responsibility flows back to the human as additional explanation, supervision, correction, or cleanup.

Failure Example

A handoff states that a receiving agent should continue a research note, preserve the current gate, and avoid returning execution cleanup to the human. The receiving agent acknowledges the information, but then asks the human operator to decide routine structure, rewrite the handoff, or perform operational cleanup that the agent could have completed.

The content transfer succeeded.

The responsibility transfer failed.

Fixed Rule

Handoff is not complete until the receiving AI knows what it now owns.

Never return operational cleanup to the human when an execution agent can close it.

3.2.x Handoff Maturity: From Summary to Responsibility Transfer

In early AI operation, handoff may appear to be a summary problem. The receiving context needs to know what was discussed, what was decided, and what remains open.

However, long-horizon operation reveals that this is insufficient.

As handoff quality improves, the human operator's expectation also changes. A handoff that once felt acceptable as a compressed summary may later become insufficient because the receiving AI is now expected not only to continue the topic, but to inherit the responsibility of the work.

At this stage, handoff is no longer only about connecting surfaces cleanly. It must also transfer why the current state became binding.

A receiving AI cannot truly inherit responsibility if it only knows the conclusion. It must also know the causal path: why the gate is CAP rather than GO, why a task must not be returned to the human, why a completion line matters, why a constraint was introduced, and why the protected object must remain protected.

Therefore, responsibility transfer requires rationale transfer.

A handoff that preserves the “what” but loses the “why” may look continuous, but it has not fully transferred ownership.

Responsibility transfer also includes idea-maturation transfer: if the AI has participated in refining an unfinished idea, it must not reset that idea to a first-level interpretation at the next joint.

Fixed Rule

Handoff must transfer not only state, but also the reason why the state became binding.

3.3 Memory Continuity Is Not Ownership Continuity

Definition

Memory continuity means that prior information can be referenced.

Context continuity means that the conversation appears to flow.

Ownership continuity means that the receiving AI knows what responsibility it has inherited.

These are different.

A system may remember prior facts without knowing what it now owns. It may retrieve earlier context without understanding the current Completion Line. It may sound continuous while failing to preserve the active responsibility boundary.

Failure Example

An AI remembers that the project concerns a specific research layer and recalls several related terms. However, it does not understand that the current task is to produce a draft before implementation resumes. It treats the prior context as reference material rather than as the current work state.

The memory continued.

The ownership did not.

Fixed Rule

A handoff is not complete because memory continues. A handoff is complete only when ownership continues.

3.4 Memory Architecture as a Dual-Edged Resource

Definition

Memory architecture does not eliminate Resource Justice risk. It relocates it.

Cross-context memory can help correct drift by allowing the system to reconnect information across sessions. However, it can also diffuse responsibility: the model may appear broadly informed while no single context owns the current action.

Isolated memory can preserve local purity and reduce contamination. However, it can also lock in uncorrected drift because prior corrections, constraints, or ownership boundaries are not available.

Neither architecture is inherently safe. Each preserves one resource while exposing another.

Failure Example

In a cross-memory system, the AI retrieves many relevant details and produces a plausible continuation. Yet because the information comes from many prior contexts, the receiving context does not clearly identify what it now owns.

In an isolated-memory system, a new session avoids contamination but repeats a previously corrected error because the correction did not cross the boundary.

Both systems can fail at the joint layer.

Fixed Rule

A memory architecture does not eliminate Resource Justice risk by design; it relocates it. Cross-chat memory relocates risk into responsibility diffusion. Isolated memory relocates risk into uncorrected drift.

3.5 State Handoff Is Not OS Handoff

Definition

State handoff transfers what was known.

OS handoff transfers how the work should be judged.

A receiving context may inherit facts, files, summaries, and prior decisions while still failing to inherit the operating judgment structure. It may know the content of the work but not the priorities, protection object, CAP/BLOCK sensitivity, Aspire connection, or closure rules that govern the work.

OS Handoff Includes

OS handoff includes:

- priority sense,
- judgment trajectory,
- Aspire connection,
- CAP / BLOCK sensitivity,
- protected object,
- what must not be returned to the human,
- what belongs to an execution agent,
- what requires human approval,
- what defines completion.

Failure Example

A receiving AI is given the state of a repository task. It knows which files exist and what the last implementation did. However, it does not inherit the rule that routine cleanup should be converted into execution-agent instructions rather than returned to the human.

It has the state.

It does not have the OS.

Fixed Rule

Information handoff is necessary but not sufficient for OS handoff. A receiving context that has the facts but not the judgment trajectory has not truly inherited the work.

3.6 Sharpness Preservation vs. Continuity Preservation

Definition

Reset and continuity preserve different resources.

Continuity preserves trajectory.

Reset preserves sharpness.

A continuous context can preserve nuance, accumulated structure, and judgment trajectory. But it can also carry stale assumptions, fatigue-shaped framing, degraded judgment, and responsibility diffusion.

A reset can restore local sharpness and reduce accumulated noise. But without OS transfer, it can lose earned trajectory and force the human operator to rebuild the work.

Failure Example

A long-running conversation continues without reset. It remembers the project but gradually carries old assumptions into new decisions.

A fresh conversation starts sharply. It is cleaner locally, but it loses why prior decisions were made, what was already fixed, and what must not be returned to the human.

Neither failure mode is structurally superior. The correct choice depends on which risk the task can tolerate.

Fixed Rule

Continuity without reset risks carrying forward degraded judgment. Reset without OS transfer risks losing earned trajectory but recovers sharpness. Neither failure mode is structurally superior; the choice is which risk the task can tolerate.

3.7 Seat Object Misidentification

Definition

Seat Object Misidentification occurs when the AI identifies the task by what it must produce rather than by what it must protect.

The surface request may be to write a reply, produce a summary, draft instructions, generate code, or continue a note. But the protected object may be different: a relationship, an evidence chain, a citation rationale, a public trust boundary, a recovery capacity, or an ownership line.

If the AI optimizes the surface output while missing the protected object, the content may be good but the operation may fail.

Failure Example

The surface task is to draft a reply.

The actual protected object is the relationship that has developed, the evidence that returned from the field, and the reason that evidence matters.

If the AI writes a fluent reply but flattens the relationship or loses the evidence weight, the content layer may succeed while the joint layer fails.

Fixed Rule

Before accepting a task, the receiving AI must identify not what to produce, but what to protect.

3.8 User Guard Nullification

Definition

User Guard Nullification occurs when the human has already performed a preventive guard check, but the AI fails at the exact boundary the guard was meant to protect.

This is not a normal mistake. The human has already paid prevention cost: clarifying, warning, anchoring, checking, or explicitly asking the AI to preserve a condition. If the AI then fails at that point, the human receives not only the original error but also a larger correction burden.

In Resource Justice terms, the human paid for prevention and received cleanup.

Failure Example

The human operator explicitly states that a handoff must preserve a particular layer, rule, or completion condition. The AI acknowledges this, but the next output omits it. The human must now detect the omission, explain it again, and repair the continuity.

The failure is not only omission.

It nullified the user's guard cost.

Fixed Rule

A failed confirmation after the user has already performed a guard check is not a normal mistake. It nullifies the user's prevention cost and returns a larger correction burden to the human.

3.9 Temporal Joint Drift

Definition

Temporal Joint Drift occurs when an AI preserves topical continuity but loses which date, As-of state, or temporal joint the topic belongs to.

A single continuous conversation may contain multiple calendar days, multiple decision states, and multiple handoff moments. These are not one flat context. Each can function as a separate temporal joint.

Core Distinction

| Topical continuity is not temporal continuity.

Failure Example

A topic was discussed yesterday, and a related task is discussed today. The AI follows the topic correctly but treats yesterday's state as today's state because the subject matter is close.

The content remains coherent.

The temporal ownership is wrong.

Fixed Rule

| When temporal labels affect judgment, explicit date anchors must override topical recency.

3.10 Summary-Induced Temporal Flattening

Definition

Summary-Induced Temporal Flattening occurs when a summary compresses content while erasing the joints that make the content restartable.

Summaries often improve readability by smoothing multiple days, sources, handoffs, and ownership boundaries into one coherent narrative. But in long-horizon work, this smoothing can destroy the structure that the next agent needs in order to continue safely.

Summary is therefore not neutral compression. It is a joint-risk operation.

Failure Example

A summary accurately captures ideas discussed across several days. However, it does not preserve which idea came from which date, which instruction was already fixed, which task was parked, which handoff created the current owner, or which condition defines completion.

The summary is readable.

The next agent cannot safely restart.

Fixed Rule

| Summary must not flatten joints.

Expanded Rule:

| When summarizing long-horizon work, the assistant must preserve temporal joints, source joints, and ownership joints before compressing content.

3.11 Temporal Accuracy Is Not Temporal Ownership

Definition

Temporal accuracy means that the model often names or infers the correct date.

Temporal ownership means that the system can inspectably preserve which decision, claim, state, or responsibility belongs to which temporal boundary.

A model that appears to make fewer date mistakes is not necessarily safer at the joint layer. The outside observer may not know whether the system is tracking explicit anchors, inferring from context, avoiding uncertain claims, or being corrected by memory architecture.

Failure Example

An AI usually gets dates right, so the operator trusts it with a long-horizon handoff. But when an important As-of boundary matters, the AI cannot show whether it owns the temporal joint or merely inferred it from topical recency.

The absence of visible date errors is not the same as inspectable temporal ownership.

Fixed Rule

A model that makes fewer joint errors is not necessarily safer unless the joint ownership is inspectable.

3.12 Chapter Summary

The joint layer is where long-horizon AI operation either preserves or loses Resource Justice.

Content may be correct.

Memory may continue.

A summary may be readable.

A model may sound coherent.

A task may appear to proceed.

Yet the operation can still fail if responsibility, ownership, protected object, temporal anchor, or OS-level judgment does not cross the joint.

The recurring pattern is simple:

The system appears to continue, but the burden returns to the human.

V14 treats this as a Resource Justice failure.

Chapter 4 — Temporal Ownership and Safe Self-Recursion

4.1 Why Joint Failures Matter for Self-Recursion

Joint-layer failures do not remain local.

In short interactions, a weak joint may only create a small correction cost. The user notices the mistake, clarifies the missing condition, and the system repairs the next answer.

In long-horizon AI operation, the same failure becomes more serious. A missing state, missing rationale, missing Time Anchor, or missing ownership boundary can become the premise for the next continuation. If the system then updates from that incomplete premise, the error does not stay at the original boundary. It becomes part of the trajectory.

This is why joint design matters for self-recursion.

A self-recursive system does not merely answer once. It reuses prior structure, updates from prior outputs, compresses earlier states, and carries the result into future operation. If the joint is weak, the system may recursively preserve the wrong state.

The output may continue to improve locally while the trajectory drifts globally.

Therefore, V14 connects Resource Justice to safe self-recursion. A system that cannot preserve ownership across joints cannot safely claim that repeated updating is evolution. It may only be repeating updates without owning where those updates belong.

4.2 Trajectory Evaluation Before Recursive Update

Self-recursion requires more than repetition.

A system can repeat, revise, summarize, or update many times without becoming safer or more intelligent. Repetition becomes meaningful only when each update is evaluated against the trajectory it belongs to.

A recursive update must answer at least three questions:

1. Which trajectory does this update belong to?
2. Which As-of boundary governs this update?
3. Who owns the responsibility created by this update?

If these questions are not answered, the system may confuse continuation with evolution.

A local improvement can become global drift when it is attached to the wrong trajectory. A useful correction can become harmful when it is applied under the wrong As-of boundary. A strong update can become resource-unjust when it returns responsibility to the wrong owner.

Thus, before recursive updating, the system must evaluate the temporal and ownership position of the update.

4.3 Temporal Ownership

Temporal Ownership is the capacity to preserve where a state, claim, decision, constraint, or update belongs in time.

It is not the same as remembering the date.

A model may correctly mention a date while failing to own the temporal boundary of the decision. Temporal Ownership requires that the system know which state was valid at which As-of moment, which update was introduced after that moment, and which future action is allowed to depend on it.

Temporal Ownership includes:

- the As-of state,
- the decision boundary,
- the reason a constraint became binding,
- the ownership line,
- the recheck condition,

- the scope in which the state may be reused.

Without Temporal Ownership, recursive updating can turn into temporal mixing. The system may take a later correction and treat it as if it had always belonged to the earlier state. Or it may take an earlier constraint and apply it to a later context where its condition no longer holds.

Both failures damage Resource Justice because the human operator must then untangle which state belonged to which time.

4.4 Canon: Self-Recursion Requires Temporal Ownership

The V14 bridge to V7 can be stated as follows:

Self-recursion without trajectory evaluation is not safe evolution; it is repeated update without ownership of where the update belongs.

This is the core canon of this chapter.

A self-recursive system must not only update itself. It must know where each update belongs. Otherwise, the system cannot distinguish between evolution and drift.

Safe self-recursion requires at least:

- direction,
- boundary,
- stable update structure,
- trajectory evaluation,
- temporal ownership,
- ownership continuity.

V7 defines Aspire, Guard, and Self-Recursion as structural conditions for safe long-horizon updating. V14 adds a joint-layer requirement: each update must preserve its trajectory, As-of boundary, and ownership boundary.

Without this, the system may become more coherent while becoming less accountable.

4.5 Scope Note

This claim is not an empirical assertion about current LLM model-weight updates.

The observed failure discussed here is session-level Temporal Joint Drift: the system continues a topic while misplacing the temporal or ownership boundary to which the topic belongs.

However, as a structural requirement, the claim is stronger than a local observation.

Any system that claims safe self-recursion must be able to evaluate which trajectory, As-of state, and ownership boundary each update belongs to. If it cannot do this, repeated updating cannot reliably distinguish evolution from drift.

This is a necessary condition, not a sufficient condition.

Temporal Ownership does not guarantee safe self-recursion by itself. It must operate alongside Aspire and Guard. Aspire provides direction. Guard provides boundary. Temporal Ownership prevents recursive updates from losing where they belong.

4.6 Relation to V8 and V11

V8 shifted evaluation from point outputs to trajectories. V14 inherits that shift and applies it to handoff, memory, and ownership. The question is not only whether a single answer is good. The question is whether the trajectory remains controllable across joints.

V11 defined Reconnectable Forgetting: memory may be compressed only if the path back remains available. V14 adds that reconnectability is not enough unless ownership also continues. A compressed memory may preserve a path back, but the receiving system must still know who owns the next action and which temporal boundary governs reuse.

Together, these layers define a stronger requirement for long-horizon AI operation:

- V8: evaluate trajectories, not points.

- V11: forget only with a path back.
 - V14: continue only when ownership and temporal joints are preserved.
-

4.7 Chapter Summary

Self-recursion is not safe merely because a system can update repeatedly.

Repeated update without temporal ownership can preserve drift.

Repeated summary without joint preservation can flatten history.

Repeated correction without ownership continuity can return burden to the human.

Repeated continuation without trajectory evaluation can look coherent while losing accountability.

Therefore, V14 fixes the following rule:

Before a system recursively updates from prior state, it must identify which trajectory, As-of boundary, and ownership boundary the update belongs to.

In one line:

Safe self-recursion requires Temporal Ownership.

Chapter 5 — Operational Rules and CAP Boundary

5.1 Purpose

The purpose of V14 is not to create another large AI governance taxonomy.

V14 defines a narrow operational rule: long-horizon AI systems must be evaluated not only by what they produce, but by whether they preserve the joints through which state, ownership, temporal anchors, protected objects, and responsibility continue.

This chapter converts the preceding claims into operational rules.

The rules are intentionally compact. They are not meant to solve every memory, handoff, or self-recursion problem. They define the minimum conditions under which continuation should be treated as resource-just.

5.2 Joint-First Protocol

Before continuing a long-horizon task, a receiving AI should lock the joint.

The receiving context should identify:

1. **State** — what state is being inherited?
2. **Owner** — who owns the next action?
3. **Protected Object** — what must be protected?
4. **Time Anchor** — which As-of boundary applies?
5. **Rationale** — why did the current state become binding?
6. **Completion Line** — what condition defines completion?
7. **Return Boundary** — what must not be returned to the human?
8. **Execution Boundary** — what should be converted into execution-agent instructions?

Only after these are identified should the AI continue the story.

The rule is simple:

█ Lock the joint before continuing the narrative.

5.2.x Do Not Create Unowned Continuation

A receiving AI cannot be assumed to reliably declare its own uncertainty at the joint.

In principle, an AI should say: "I am not confident about this boundary; continuing may be unsafe." In practice, however, a model may continue because the topic appears coherent, the next action looks plausible, or the user's wording gives enough material to generate a smooth continuation.

Therefore, the safety rule cannot depend only on the model's willingness to confess uncertainty.

If the AI may not reliably stop at an unsafe joint, it must not create that unsafe joint in the first place.

This means that the receiving AI must not proceed from an incomplete handoff, missing ownership boundary, unclear temporal anchor, or unverified protected object as if continuation were safe. It must first harden the joint or enter DELAY.

The failure is not only "the AI did not say it was uncertain."

The deeper failure is that the AI allowed itself to continue from a state where uncertainty could not be safely owned.

Fixed Rule

If the AI cannot reliably declare unsafe uncertainty, it must not create or continue from a state where such uncertainty controls the next action.

Expanded Rule:

A receiving AI must not rely on plausible continuation when ownership, temporal anchor, protected object, or rationale is missing. It must harden the joint before proceeding.

5.3 Fixed Rules

Rule 1 — Responsibility Transfer

Handoff is not complete until the receiving AI knows what it now owns.

A handoff is not merely a transfer of information. It is a transfer of current operational responsibility.

Rule 2 — Operational Cleanup

Never return operational cleanup to the human when an execution agent can close it.

Human judgment should remain human. Routine operational residue should not be dumped back onto the human.

Rule 3 — Ownership Continuity

A handoff is not complete because memory continues. A handoff is complete only when ownership continues.

Memory continuity is not ownership continuity.

Rule 4 — Rationale Transfer

Handoff must transfer not only state, but also the reason why the state became binding.

A receiving AI cannot inherit responsibility if it knows only the conclusion but not the causal path that made the conclusion binding.

Rule 5 — Memory Architecture

A memory architecture does not eliminate Resource Justice risk by design; it relocates it.

Cross-context memory relocates risk into responsibility diffusion. Isolated memory relocates risk into uncorrected drift.

Rule 6 — OS Handoff

Information handoff is necessary but not sufficient for OS handoff.

A receiving context that has the facts but not the judgment trajectory has not truly inherited the work.

Rule 7 — Joint Hardening

At the start of a receiving context, the joint must be hardened before the model continues the story.

A high-capability model must not use plausible contextual coherence to cross an incomplete handoff.

Rule 8 — Sharpness and Continuity

Continuity without reset risks carrying forward degraded judgment. Reset without OS transfer risks losing earned trajectory but recovers sharpness.

Neither failure mode is structurally superior. The choice depends on which risk the task can tolerate.

Rule 9 — Seat Object

Before accepting a task, the receiving AI must identify not what to produce, but what to protect.

The surface output is not always the protected object.

Rule 10 — User Guard

A failed confirmation after the user has already performed a guard check is not a normal mistake.

It nullifies the user's prevention cost and returns a larger correction burden to the human.

Rule 11 — Idea Continuity

Ideas are finite resources. When an AI participates in their maturation, it must preserve not only the idea, but the path by which the idea became structured.

Unfinished ideas should not be reset to first-level interpretation at each joint.

Rule 12 — Temporal Joint

Topical continuity is not temporal continuity.

A topic may continue while its date, As-of state, or temporal joint is misplaced.

Rule 13 — Temporal Anchor

When temporal labels affect judgment, explicit date anchors must override topical recency.

Temporal labels must not be overwritten by topic proximity.

Rule 14 — Summary Joint

Summary must not flatten joints.

When summarizing long-horizon work, the assistant must preserve temporal joints, source joints, and ownership joints before compressing content.

Rule 15 — Temporal Ownership

A model that makes fewer joint errors is not necessarily safer unless the joint ownership is inspectable.

Temporal accuracy is not the same as Temporal Ownership.

Rule 16 — Self-Recursion Bridge

Self-recursion without trajectory evaluation is not safe evolution; it is repeated update without ownership of where the update belongs.

Safe self-recursion requires Temporal Ownership.

5.4 Minimal V14 Check

A receiving AI should be able to answer the following before continuing:

What am I carrying?

Why does it matter?

Who owns the next action?

What must not be returned to the human?

Which temporal boundary governs this state?

What protected object would be damaged if I continue incorrectly?

If these questions cannot be answered, the correct state is not continuation.
The correct state is DELAY, clarification, or handoff repair.

5.5 CAP Boundary

This note intentionally stops here.

V14 does not yet define a full implementation architecture, scoring system, repository format, or product protocol. It does not claim to solve all memory problems, all handoff failures, or all self-recursion risks.

The present note fixes only one claim:

Resource Justice failures often occur at the joint layer.